Robotik Mühendisliği Serisi — Cilt 2

Yazılım: Linux, Git, C++, CMake, Python, ROS 2

Robot yazılımının mühendislik araçları

Türkçe–İngilizce karma konu anlatımı
teori · sayısal örnekler · çalışan Python ve C++ kodları

Odak: bacaklı ve humanoid robotlar

31 Ekim 2026

İçindekiler

Bu Cilt Hakkında **1**

1 Linux ve Kabuk **2**

1.1 Hayatta kalma komutları . . . 2

1.2 Yönlendirme, boru, süreçler . . . 2

1.3 Donanımla konuşmak . . . 2

2 Git ve GitHub İş Akışı **3**

2.1 Temel döngü . . . 3

2.2 Git içeride nasıl çalışır? . . . 3

2.3 Dallar ve çekme istekleri . . . 3

2.4 İyi alışkanlıklar . . . 4

3 C'den C++'a: RAII, Akıllı İşaretçiler, Taşıma **5**

3.1 Derleme, bağlama ve bellek: alttaki model . . . 5

4 Şablonlar, STL ve Lambda'lar **8**

5 Eşzamanlılık **11**

6 Gerçek Zamanlı Kontrol Döngüsü **13**

7 CMake, Proje Yapısı ve Testler **15**

7.1 Yapı . . . 15

7.2 Derleme ve test . . . 17

8 Robotik için Python **19**

8.1 Ortam . . . 19

8.2 Alışkanlıklar ve tuzaklar . . . 19

8.3 pytest . . . 20

9 SB-1'in MuJoCo Modeli (MJCF) **22**

10 ROS 2 **25**

10.1 Temel komutlar . . . 25

10.2 SB-1 IK düğümü 25
10.3 ros2_control ve hızlı döngü 26

11 Sorular **27**

12 Cevaplar **29**

Bu Cilt Hakkında

Diğer ciltler "ne hesaplanır" sorusunu cevaplıyor; Cilt 2 "bu hesaplar nasıl *güvenilir yazılıma* dönüşür" sorusunu. Bir robotik mühendisinin günlük işinin büyük kısmı burada: Linux terminali, Git, C++ ile gerçek zamanlı kod, derleme sistemi, test, Python ile analiz, simülasyon modeli ve ROS 2.

C biliyorsun; bu yüzden C++ bölümleri "C'de bunu nasıl yapardın, C++ neyi otomatikleştiriyor" karşılaştırmasıyla ilerliyor. Örneklerin hepsi seride tanıdık olan şeyler: SB-1'in bacak kinematiği, Cilt 4A'daki PID, servo okumaları.

Neler denendi, neler denenmedi: Bölüm 3–8'deki tüm C++ ve Python kodu bu belgede derlendi/çalıştırıldı, çıktılar gerçek. `sb1_core` projesi CMake ile kuruldu ve 7 GoogleTest testi geçti; Python sürümü pytest'te 27 test geçti. **ROS 2 kodu (Bölüm 10) ve MJCF modelinin MuJoCo'da yüklenmesi bu ortamda denenmedi** (ikisi de kurulu değil); dosyalar dikkatle yazıldı ama ilk çalıştırmada küçük düzeltmeler gerekebilir. Bunu açıkça işaretledim.

Bölüm	Konu	Kod
1	Linux ve kabuk	–
2	Git ve GitHub iş akışı	–
3	C'den C++'a: RAII, akıllı işaretçiler, taşıma	`y2_raii.cpp`
4	Şablonlar, STL, lambda	`y3_stl.cpp`
5	Eşzamanlılık: thread, mutex, atomic	`y4_threads.cpp`
6	Gerçek zamanlı kontrol döngüsü ve titreşim (jitter)	`y5_rtloop.cpp`
7	CMake, kütüphane yapısı, GoogleTest	`proje/sb1_core`
8	Robotik için Python: vektörleştirme, tuzaklar, pytest	`y7_python.py`, `proje/py_sb1`
9	SB-1'in MuJoCo modeli (MJCF) koddan	`y8_mjcf.py`
10	ROS 2: düğüm, konu, parametre, launch, ros2_control	`proje/ros2_sb1`
11–12	Sorular ve cevaplar	–

Staj dönemi için: telefon sürümü (Kitapçık 2) kodsuz ama kabuk komutları duruyor; 22 soru-cevap var. Bu cildi telefondan okurken her bölümün "kendine sor" kısmını not defterine yaz; bilgisayara kavuşunca ilk iş o komutları denemek.

Bölüm 1

Linux ve Kabuk

Robotların neredeyse tamamı Linux üzerinde çalışır (ROS 2'nin birincil platformu Ubuntu). Windows'ta WSL2 ile başlayabilirsin; ama gerçek zamanlı ve donanım işleri için doğrudan Ubuntu kurulumu daha az sorun çıkarır.

1.1 Hayatta kalma komutları

```
pwd; ls -la; cd ~/robot; cd ..          # neredeyim, ne var, git, geri
mkdir -p proje/src; cp -r a b; mv a b; rm -r eski   # dikkat: rm geri alınmaz
cat dosya; less uzun.log; head -n 20 x; tail -f log   # oku, canlı izle
grep -rn "kp" src/          # klasörde metin ara (satır numarasıyla)
find . -name "*.cpp"         # dosya ara
chmod +x betik.sh; ./betik.sh # çalıştırılabilir yap, çalıştır
sudo apt install build-essential cmake git   # paket kur
```

1.2 Yönlendirme, boru, süreçler

```
./robot_log > log.txt 2>&1          # stdout ve stderr dosyaya
python3 analiz.py < veri.csv | sort | uniq -c   # boru (pipe): çıktı -> girdi
ps aux | grep controller; kill -INT 1234     # süreci bul, Ctrl+C eşdeğeri gönder
htop                                 # CPU/bellek canlı
export ROS_DOMAIN_ID =7; echo $PATH   # ortam değişkenleri
```

1.3 Donanımla konuşmak

```
ls /dev/ttyUSB* /dev/ttyACM*        # Arduino genelde /dev/ttyACM0
sudo usermod -aG dialout $USER       # seri porta sudo'suz erişim (çıkış-giriş yap)
dmesg | tail                          # USB takınca çekirdek ne dedi?
screen /dev/ttyACM0 115200            # seri terminal (çıkış: Ctrl+A, K)
ssh pi@192.168.1.20; scp derleme.tar pi@192.168.1.20:~   # robottaki bilgisayara bağlan, dosya at
```

Kendine sor

Bir program "Permission denied" diyorsa üç olasılık: dosya çalıştırılabilir değil (`chmod +x`), cihaz dosyasına erişim yok (`dialout` grubu), ya da gerçekten yönetici yetkisi gerek (`sudo`; ama ROS düğümlerini asla `sudo` ile çalıştırma).

Bölüm 2

Git ve GitHub İş Akışı

Git, kodun zaman makinesidir: her *commit* projenin bir anlık görüntüsü. Lisansüstü başvurularında GitHub profilin, CV'nin yazılı olmayan yarısıdır.

2.1 Temel döngü

```
git init; git clone https://github.com/kullanici/sb1.git
git status                    # ne değişti?
git diff                      # satır satır ne değişti?
git add src/leg.cpp           # sahneye al (sadece istediğin dosyalar)
git commit -m "leg: IK erişim dışı durumu nullopt döndürüyor"
git log --oneline --graph     # geçmiş
git push origin main          # GitHub'a gönder
```

2.2 Git içeride nasıl çalışır?

Komutları ezberlemek yerine modeli anlamak, hataları çözmeyi kolaylaştırır. Git bir **içerik adresli** veritabanıdır; üç tür nesne saklar:

- **blob:** bir dosyanın içeriği. Adı, içeriğin SHA özetidir (40 onaltılık karakter). Aynı içerik = aynı ad: iki dosya aynıysa bir kez saklanır.
- **tree:** bir klasör: (isim, blob veya tree özeti) listesi.
- **commit:** kök tree'nin özeti + ebeveyn commit(ler)in özeti + yazar, tarih, mesaj.

Bir commit'in özeti, ebeveyninin özetini içerir; dolayısıyla geçmişin herhangi bir yerinde tek bir bayt değişse sonraki tüm özetler değişir (zincirleme bütünlük). **Dal** sadece bir commit özetini tutan küçük bir dosyadır (`.git/refs/heads/main`); `HEAD` şu an hangi dalda olduğunu gösterir. Commit atmak: yeni commit nesnesi oluştur, dal dosyasını ona güncelle. Bu yüzden dal açmak bedavadır ve "kaybolan" commit'ler genelde hâlâ veritabanındadır, sadece onları gösteren bir isim kalmamıştır (`git reflog` onları bulur). **Birleştirme (merge):** iki ebeveynli bir commit; **rebase:** commit'leri yeni bir tabanın üstüne *yeniden oluşturmak* (yeni özetler, bu yüzden paylaşılmış dalı rebase etme).

2.3 Dallar ve çekme istekleri

```
git switch -c ozellik/dcm-planner     # yeni dal
# ... değişiklikler, commit'ler ...
git push -u origin ozellik/dcm-planner
# GitHub'da Pull Request aç, CI testleri çalışsın, incele, birleştir
git switch main; git pull             # birleştirilmiş main'i al
git rebase main                       # (dal eskidiyse) dalını main'in üstüne taşı
```

2.4 İyi alışkanlıklar

- **Küçük, anlamlı commit'ler.** Mesaj: "ne" ve "neden". "düzeltme" kötü; "pid: türevi ölçümden al (basamakta tekme vardı)" iyi.
- `.gitignore`: `build/`, `__pycache__/`, `*.o`, büyük veri ve videolar depoya girmez.
- **Asla** şifre, API anahtarı veya kişisel veri commit'leme; commit'lendiyse geçmişten silmek zordur, anahtarı iptal et.
- Her depoya bir `README.md`: ne yapar, nasıl derlenir, bir görsel/video, sonuçlar. Bir hocanın 30 saniyede anlayabileceği kadar.
- GitHub Actions ile her push'ta testleri çalıştır (Bölüm 7'deki `ctest`).

En sık felaket

`git reset --hard` ve `git push --force` geri alınamayan iş kaybı yaratabilir. Emin değilsen önce `git stash` veya yeni bir dal aç. Kaybolan commit'ler çoğu zaman `git reflog` ile bulunur.

Bölüm 3

C'den C++'a: RAII, Akıllı İşaretçiler, Taşıma

C'de her `malloc`'un bir `free`'si, her `fopen`'ın bir `fclose`'u olmalı ve fonksiyonun *her* çıkış yolunda (erken `return`, hata) bunu hatırlaman gerekir. C++'ın en önemli fikri bunu derleyiciye devretmek:

RAII (Resource Acquisition Is Initialization): kaynağı kurucu (constructor) alır, yıkıcı (destructor) bırakır. Nesne kapsamdan çıktığı an, nasıl çıkarsa çıksın, yıkıcı çağrılır. Seri port, dosya, mutex kilidi, GPU belleği, ROS düğümü: hepsi böyle yönetilir.

- `std::unique_ptr<T>`: tek sahip; kopyalanamaz, `std::move` ile devredilir. Varsayılan seçim.
- `std::shared_ptr<T>`: paylaşılan sahiplik, referans sayacı; son sahip bırakınca silinir. ROS 2 mesajları ve düğümleri böyle dolaşır. Bedeli: sayaç (atomik işlem).
- Ham işaretçi (`T*`) ve referans (`T&`): "sahip olmadan bakmak" için. `new`/`delete`'i elle yazman neredeyse hiç gerekmez.
- **Taşıma**: `T(T&&)` kurucusu kaynağı kopyalamak yerine "çalar" ve kaynağı boş bırakır. Büyük vektörler ve kopyalanamayan kaynaklar (port) böyle verimli dolaşır.
- **Beşin kuralı**: kaynağı elle yöneten bir sınıf yazıyorsan yıkıcı, kopya kurucu/atama, taşıma kurucu/atama beşini birlikte düşün. Çoğu zaman sıfır kuralı daha iyi: üyeleri zaten RAII olan türlerden (vector, unique_ptr) kur, hiçbirini yazma.

3.1 Derleme, bağlama ve bellek: alttaki model

Derleme üç adım: (1) *önişlemci* `#include`'ları metin olarak yapıştırır, `#define`'ları açar; (2) *derleyici* her `.cpp`'yi ayrı ayrı makine koduna (`.o`) çevirir; başka dosyadaki fonksiyonları sadece *bildirimden* (başlıktan) bilir; (3) *bağlayıcı* (linker) `.o`'ları ve kütüphaneleri birleştirip her çağrıyı gerçek adrese bağlar. "undefined reference" hatası bağlayıcıdan gelir (bildirim var, tanım yok); "multiple definition" de (aynı tanım iki `.o`'da: başlıkta `inline` olmadan fonksiyon gövdesi).

Yığın (stack) ve öbek (heap). Yerel değişkenler yığında: fonksiyon girerken yığın işaretçisi kaydırılır, çıkarken geri alınır; neredeyse bedava ve otomatik. `new`/`malloc` öbekten yer ister: işletim sistemi/ayırıcı ile konuşur, süresi belirsiz, ve birinin `delete` etmesi gerekir. RAII'nin hilesi: öbekteki kaynağı *yığındaki* bir nesneye bağlamak; yığın nesnesinin ömrü kapsamla bittiğinde yıkıcısı öbeği temizler. `std::unique_ptr` derlendiğinde ham bir işaretçiyle aynı boyutta ve hızdadır (sıfır maliyetli soyutlama); farkı sadece yıkıcıda `delete` çağırması ve kopyalamanın yasak olması.

Taşıma ne yapar? `std::vector` içeride üç işaretçi tutar (başlangıç, son, kapasite). Kopyalamak n elemanı yeni bir bloğa kopyalamaktır ($\mathcal{O}(n)$); taşımak üç işaretçiyi kopyalayıp kaynağınkileri `nullptr` yapmaktır ($\mathcal{O}(1)$). `std::move` hiçbir şeyi taşımaz; sadece "bu nesneyi artık kullanmayacağım, kaynaklarını alabilirsin" diye türünü değiştirir (sağ değer referansı `T&&`), ve taşıma kurucusu seçilir.

C++

```cpp
// y2_raii.cpp — C'den C++'a: RAII, akıllı işaretçiler, taşıma (move) semantiği
#include <cstdio>
#include <memory>
#include <utility>
#include <vector>

// Bir donanım kaynağını temsil eden sınıf (ör. seri port). Kurucu açar, yıkıcı KAPATIR: RAII.
class SerialPort {
public:
    explicit SerialPort(const char* name) : name_(name) { std::printf("  [aç]    %s\n", name_); }
    ~SerialPort() { if (name_) std::printf("  [kapat] %s\n", name_); }
    SerialPort(const SerialPort&) = delete;                 // kopyalanamaz: aynı portu iki nesne kapatmasın
    SerialPort& operator=(const SerialPort&) = delete;
    SerialPort(SerialPort&& o) noexcept : name_(std::exchange(o.name_, nullptr)) {}   // taşınabilir: sahiplik devri
    void write(int v) const { std::printf("  %s <- %d\n", name_, v); }
private:
    const char* name_;
};

void erken_cikis(bool hata) {
    SerialPort p("/dev/ttyUSB0");
    p.write(90);
    if (hata) { std::printf("  hata! erken return\n"); return; }   // C'de burada close() unutulurdu
    p.write(45);
}                                                           // her yoldan çıkışta yıkıcı çağrılır

struct Servo { int id; explicit Servo(int i) : id(i) {} ~Servo() { std::printf("  servo %d silindi\n", id); } };

int main() {
    std::printf("1) RAII, normal ve erken çıkış:\n");
    erken_cikis(false);
    erken_cikis(true);

    std::printf("2) unique_ptr: tek sahip, otomatik delete\n");
    {
        auto s = std::make_unique<Servo>(1);
        std::unique_ptr<Servo> t = std::move(s);                    // sahiplik t'ye geçti
        std::printf("  s boş mu? %s, t->id = %d\n", s ? "hayır" : "evet", t->id);
    }                                                           // t kapsamdan çıkınca servo 1 silinir

    std::printf("3) shared_ptr: paylaşılan sahiplik, sayaç\n");
    std::shared_ptr<Servo> a = std::make_shared<Servo>(2);
    {
        std::shared_ptr<Servo> b = a;
        std::printf("  sayaç = %ld\n", a.use_count());
    }
    std::printf("  sayaç = %ld\n", a.use_count());
    a.reset();                                                  // son sahip bıraktı: silinir

    std::printf("4) vector içinde taşınabilir kaynaklar\n");
    std::vector<SerialPort> ports;
    ports.reserve(2);                                           // yeniden boyutlanırken taşıma olmasın
    ports.emplace_back("/dev/ttyACM0");
    ports.push_back(SerialPort("/dev/ttyACM1"));                // geçici nesne taşınır (kopya YOK)
```

```
    std::printf("  main bitiyor\n");
    return 0;                                                    // vektör yok olur, portlar kapanır
}
```

Çıktı:

```
1) RAII, normal ve erken çıkış:
  [aç]    /dev/ttyUSB0
  /dev/ttyUSB0 <- 90
  /dev/ttyUSB0 <- 45
  [kapat] /dev/ttyUSB0
  [aç]    /dev/ttyUSB0
  /dev/ttyUSB0 <- 90
  hata! erken return
  [kapat] /dev/ttyUSB0
2) unique_ptr: tek sahip, otomatik delete
  s boş mu? evet, t->id = 1
  servo 1 silindi
3) shared_ptr: paylaşılan sahiplik, sayaç
  sayaç = 2
  sayaç = 1
  servo 2 silindi
4) vector içinde taşınabilir kaynaklar
  [aç]    /dev/ttyACM0
  [aç]    /dev/ttyACM1
  main bitiyor
  [kapat] /dev/ttyACM0
  [kapat] /dev/ttyACM1
```

Çıktıda dikkat: erken `return`'de de port kapandı; `unique_ptr` kapsam sonunda servoyu sildi; `shared_ptr` sayacı 2'den 1'e düştü ve `reset` ile silindi; vektördeki portlar `main` bitince kapandı ve hiçbiri iki kez kapanmadı (taşınan nesnenin `name_`'i `nullptr` yapıldı).

C ile karşılaştırma

C'de `struct` + `port_open()`/`port_close()` fonksiyonları yazardın ve `goto cleanup` kalıbı kullanırdın. C++'ta `class` aynı `struct`'tır (varsayılan erişimi `private`); fark, kurucu/yıkıcının dil tarafından garanti edilmesi. `= delete` bir fonksiyonu yasaklar: kopyalamaya çalışan kod derlenmez, hata çalışma zamanına kalmaz.

Bölüm 4

Şablonlar, STL ve Lambda'lar

Şablon (template): türden bağımsız kod. `template <typename T, std::size_t N> class RingBuffer` bir kez yazılır; `RingBuffer<double,5>` ve `RingBuffer<int,100>` için derleyici ayrı, tam hızda kod üretir. Eigen'in `Matrix<double,3,3>`'ü (Cilt 1, 5) de böyle çalışır.

STL kapları: `std::vector` (dinamik dizi, varsayılan seçim), `std::array` (sabit boyut, yığında), `std::map`/`std::unordered_map` (anahtar–değer), `std::deque`, `std::queue`. **Algoritmalar:** `sort`, `nth_element`, `count_if`, `accumulate`, `find_if`, `transform`. **Lambda:** yerinde tanımlanan isimsiz fonksiyon, `[yakalama](parametreler){gövde}`; `[&]` çevredeki değişkenleri referansla, `[ =]` kopyayla yakalar.

Şablon nasıl çalışır? Şablon bir kod değil, kod *üretme tarifidir*: `RingBuffer<double,5>` kullanıldığı anda derleyici `T =double`, `N =5` koyup sınıfı yeniden yazar ve derler (örnekleme, instantiation). Bu yüzden şablon kodu başlıkta durur (her kullanan dosyanın tarifi görmesi gerek) ve hata mesajları uzundur. Kazanç: `N` derleme anında bilindiği için dizi yığında, döngüler açılabilir; çalışma anında sıfır ek maliyet.

`vector::push_back` neden "ortalama" $\mathcal{O}(1)$? Kapasite dolunca vektör kapasiteyi (tipik olarak) ikiye katlar ve tüm elemanları yeni bloğa taşır. n ekleme boyunca yapılan toplam kopya: $1 + 2 + 4 + \cdots + n/2 < n$. Yani n eklemenin toplam maliyeti $< 2n$: eleman başına sabit (amortize). Ama *tek bir* ekleme $\mathcal{O}(n)$ sürebilir: gerçek zamanlı döngüde sorun tam olarak bu (ortalama değil, en kötü durum önemli). `reserve(n)` kapasiteyi baştan ayırır.

`nth_element` neden `sort`'tan hızlı? Medyan için tam sıralama ($\mathcal{O}(n \log n)$) gerekmez; quickselect bir pivotla diziyi böler ve sadece medyanın bulunduğu yarıda devam eder: ortalama $n + n/2 + n/4 + \cdots = 2n$, yani $\mathcal{O}(n)$.

C++

```cpp
// y3_stl.cpp — şablonlar, STL kapları ve algoritmaları, lambda'lar: robotik veri işleme örnekleri
#include <algorithm>
#include <array>
#include <cstdio>
#include <map>
#include <numeric>
#include <string>
#include <vector>

// Sabit boyutlu halka tampon (ring buffer): gerçek zamanlı kodda bellek ayırmadan son N örneği tutar
template <typename T, std::size_t N>
class RingBuffer {
public:
    void push(const T& v) { data_[head_] = v; head_ = (head_ + 1) % N; if (size_ < N) ++size_; }
    std::size_t size() const { return size_; }
    T median() const {                                  // medyan filtresi: tek aykırı ölçümü yok eder
        std::array<T, N> tmp = data_;
        auto mid = tmp.begin() + size_ / 2;
        std::nth_element(tmp.begin(), mid, tmp.begin() + size_);
        return *mid;
    }
```

```cpp
    T mean() const { return std::accumulate(data_.begin(), data_.begin() + size_, T{}) / T(size_); }
private:
    std::array<T, N> data_{};
    std::size_t head_ = 0, size_ = 0;
};

// Genel (generic) kırpma fonksiyonu: her sayısal tür için derleyici ayrı sürüm üretir
template <typename T>
constexpr T clamp_to(T v, T lo, T hi) { return v < lo ? lo : (v > hi ? hi : v); }

struct Joint { std::string name; double pos, limit_lo, limit_hi; };

int main() {
    // 1) Gürültülü bir potansiyometre okuması, arada bir sıçrama (ör. kablo teması)
    std::vector<double> raw = {1.00, 1.01, 0.99, 1.02, 3.50, 1.00, 0.98, 1.01, 1.00, -2.0, 1.02};
    RingBuffer<double, 5> rb;
    std::printf("ham    ortalama  medyan\n");
    for (double r : raw) { rb.push(r); std::printf("%5.2f   %6.3f   %6.3f\n", r, rb.mean(),
rb.median()); }

    // 2) Lambda ile sıralama ve arama: eklem sınırı ihlali
    std::vector<Joint> joints = {{"sol_kalca", 0.4, -1.0, 1.0}, {"sol_diz", 2.6, 0.0, 2.4},
                                 {"sag_kalca", -1.2, -1.0, 1.0}, {"sag_diz", 1.1, 0.0, 2.4}};
    auto ihlal = [](const Joint& j) { return j.pos < j.limit_lo || j.pos > j.limit_hi; };
    std::printf("sınır ihlali olan eklem sayısı: %ld\n", std::count_if(joints.begin(), joints.end(),
ihlal));
    for (auto& j : joints) if (ihlal(j)) {
        double old = j.pos; j.pos = clamp_to(j.pos, j.limit_lo, j.limit_hi);
        std::printf("  %-10s %5.2f -> %5.2f\n", j.name.c_str(), old, j.pos);
    }
    std::sort(joints.begin(), joints.end(), [](const Joint& a, const Joint& b) { return a.pos > b.pos;
});
    std::printf("konuma göre azalan: ");
    for (const auto& j : joints) std::printf("%s ", j.name.c_str());
    std::printf("\n");

    // 3) map: isimden indise (ROS'taki joint_states mesajı isim listesiyle gelir)
    std::map<std::string, int> index;
    for (int i = 0; i < int(joints.size()); ++i) index[joints[i].name] = i;
    std::printf("sag_diz indisi = %d\n", index.at("sag_diz"));
    std::printf("clamp_to<int>(300, 0, 180) = %d\n", clamp_to(300, 0, 180));
    return 0;
}
```

Çıktı:

```
ham    ortalama  medyan
 1.00    1.000    1.000
 1.01    1.005    1.010
 0.99    1.000    1.000
 1.02    1.005    1.010
 3.50    1.504    1.010
 1.00    1.504    1.010
 0.98    1.498    1.000
 1.01    1.502    1.010
 1.00    1.498    1.000
-2.00    0.398    1.000
 1.02    0.402    1.000
sınır ihlali olan eklem sayısı: 2
  sol_diz     2.60 ->  2.40
  sag_kalca  -1.20 -> -1.00
```

```
konuma göre azalan: sol_diz sag_diz sol_kalca sag_kalca
sag_diz indisi = 1
clamp_to<int>(300, 0, 180) = 180
```

Medyan filtre dersi: 3.50 ve −2.00 sıçramaları ortalamayı 1.50 ve 0.40'a çekti; medyan her zaman 1.00 civarında kaldı. Potansiyometre ve kızılötesi mesafe sensörlerinde tek örneklik bozulmalara karşı ucuz ve etkili (bedeli: biraz gecikme; Cilt 3 Bölüm 4).

Gerçek zamanlı kodda STL

`std::vector::push_back`, `std::map` ekleme ve `std::string` birleştirme *bellek ayırabilir*; 1 kHz döngüde bu, öngörülemeyen gecikme demektir. Kuralı: döngüden önce `reserve` et veya sabit boyutlu yapılar (`std::array`, halka tampon) kullan; döngü içinde `new`, `malloc`, dosya/konsol G/Ç yapma.

Bölüm 5

Eşzamanlılık

Bir robotta aynı anda birkaç iş yürür: sensör okuma, durum kestirimi, kontrol, haberleşme, kayıt. Bunlar ayrı **iş parçacıklarında** (thread) çalışır ve veri paylaşır. Paylaşılan veriye iki iş parçacığı aynı anda, en az biri yazarak erişirse **veri yarışı** (data race) olur: C++'ta bu tanımsız davranıştır; program "çoğu zaman" çalışıp bir gün robotu düşürür.

Araçlar:

- `std::mutex` + `std::lock_guard` (RAII kilit): kritik bölgeye tek iş parçacığı.
- `std::atomic<T>`: tek bir değişken için kilitsiz, bölünmez işlemler (bayraklar, sayaçlar).
- `std::condition_variable`: "veri gelene kadar uyu" (üretici–tüketici).
- Gerçek zamanlı döngüde kilit bile risklidir (düşük öncelikli iş parçacığı kilidi tutarken yüksek öncelikli bekler: *öncelik terslemesi*). Çözümler: kilitsiz tek-yazar/tek-okur kuyruklar, çift tampon; `ros2_control`'ün `realtime_tools` paketi bu kalıpları hazır verir.

Yarış makine düzeyinde. `++x` tek bir satır ama işlemci için üç adım: belleği yazmaca *oku*, *bir ekle*, belleğe *yaz*. İki çekirdek aynı anda okursa ikisi de 0 görür, ikisi de 1 yazar: bir artırma kaybolur. **Mutex**, kritik bölgeye aynı anda tek iş parçacığının girmesini sağlar (bekleyen iş parçacığı uyutulabilir: bu yüzden pahalı ve gerçek zamanlı döngüde riskli). **Atomic**, işlemcinin tek bir bölünmez komutunu kullanır (x86'da `lock add`): üç adım arada kimse araya giremeden yapılır. Ek bir incelik, **bellek sıralaması**: derleyici ve işlemci, tek iş parçacığında sonucu değiştirmeyen yeniden sıralamalar yapabilir; başka bir iş parçacığı bu sırayı görebilir. `std::atomic`'in varsayılanı (`memory_order_seq_cst`) herkesin aynı sırayı görmesini garanti eder; `relaxed` sadece bölünmezliği (sayaç için yeterli, bir veriyi "hazır" bayrağıyla yayınlamak için yetmez).

C++

```cpp
// y4_threads.cpp — iş parçacıkları: veri yarışı, mutex, atomic, üretici-tüketici
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdio>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

int main() {
    const int N = 200000;
    // 1) Veri yarışı: iki iş parçacığı korumasız aynı sayaca yazıyor -> TANIMSIZ DAVRANIŞ
    long racy = 0;
    {
        auto f = [&] { for (int i = 0; i < N; ++i) { long t = racy; std::this_thread::yield(); racy = t + 1; if (i > 2000) break; } };
        std::thread t1(f), t2(f); t1.join(); t2.join();
    }
```

```cpp
    std::printf("korumasız sayaç: %ld (beklenen %d) -> güncellemeler kayboldu\n", racy, 2 * 2002);

    // 2) mutex ile
    long safe = 0; std::mutex m;
    {
        auto f = [&] { for (int i = 0; i < N; ++i) { std::lock_guard<std::mutex> lk(m); ++safe; } };
// RAII kilit
        std::thread t1(f), t2(f); t1.join(); t2.join();
    }
    std::printf("mutex ile: %ld (beklenen %d)\n", safe, 2 * N);

    // 3) atomic ile (tek bir değişken için kilitsiz)
    std::atomic<long> at{0};
    {
        auto f = [&] { for (int i = 0; i < N; ++i) at.fetch_add(1, std::memory_order_relaxed); };
        std::thread t1(f), t2(f); t1.join(); t2.join();
    }
    std::printf("atomic ile: %ld\n", at.load());

    // 4) Üretici-tüketici: sensör iş parçacığı ölçüm üretir, kontrol iş parçacığı tüketir
    std::queue<int> q; std::mutex qm; std::condition_variable cv; bool done = false;
    std::thread producer([&] {
        for (int k = 0; k < 5; ++k) {
            std::this_thread::sleep_for(std::chrono::milliseconds(2));
            { std::lock_guard<std::mutex> lk(qm); q.push(100 + k); }
            cv.notify_one();
        }
        { std::lock_guard<std::mutex> lk(qm); done = true; }
        cv.notify_one();
    });
    std::vector<int> got;
    std::thread consumer([&] {
        std::unique_lock<std::mutex> lk(qm);
        while (true) {
           cv.wait(lk, [&] { return !q.empty() || done; });          // sahte uyanmalara karşı koşullu
bekle
            while (!q.empty()) { got.push_back(q.front()); q.pop(); }
            if (done) break;
        }
    });
    producer.join(); consumer.join();
    std::printf("tüketici aldı:");
    for (int v : got) std::printf(" %d", v);
    std::printf("\n");
    return 0;
}
```

Çıktı:

```
korumasız sayaç: 2002 (beklenen 4004) -> güncellemeler kayboldu
mutex ile: 400000 (beklenen 400000)
atomic ile: 400000
tüketici aldı: 100 101 102 103 104
```

Korumasız sayaçta 4004 artırmadan sadece 2002'si kaldı (bu koşuda; sonuç her çalıştırmada değişebilir, ki sorun da tam olarak bu). Mutex ve atomic her seferinde doğru 400 000'i verdi. Teşhis için: `g++ -fsanitize =thread` (ThreadSanitizer) yarışı çalışırken yakalar.

Bölüm 6

Gerçek Zamanlı Kontrol Döngüsü

Cilt 4A Bölüm 3: gecikme ve titreşim (jitter) faz payını yer. Bir kontrol döngüsünün iki özelliği önemli: **ortalama periyot** doğru mu ve **en kötü durum** (max, p99) ne kadar kötü. "Gerçek zamanlı" hızlı demek değil, *öngörülebilir* demektir.

İki döngü kalıbı:

- `work(); sleep_for(1ms);` — periyot = iş süresi + 1 ms + uyanma gecikmesi: **kayar**.
- `next += 1ms; sleep_until(next);` — mutlak zaman çizelgesine göre: hatalar birikmez.

Kaymanın hesabı. `sleep_for` ile bir döngü periyodu $T = t_{\text{iş}} + t_{\text{uyku}} + t_{\text{uyanma}}$; t_{uyanma} işletim sisteminin iş parçacığını uyandırma gecikmesi (ortalama yüzlerce µs olabilir). N döngüde toplam $N\bar{T}$, yani hata her döngüde birikir. `sleep_until` ile k. uyanma hedefi $t_0 + k\Delta$: her döngünün uyanma gecikmesi o döngünün *içinde* kalır, sonrakine aktarılmaz (bir sonraki hedef mutlak zamandan hesaplanır). Ortalama periyot tam Δ, sadece her periyot etrafında titreşim kalır. Koşul: iş + gecikme $< \Delta$; aşılırsa döngü geride kalır ve bir sonraki `sleep_until` hemen döner (yakalamaya çalışır).

C++

```cpp
// y5_rtloop.cpp — 1 kHz kontrol döngüsü: sleep_for (kayan) ve sleep_until (sabit tempolu) karşılaştırması
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdio>
#include <thread>
#include <vector>
using Clock = std::chrono::steady_clock;
using namespace std::chrono_literals;

volatile double sink = 0;
void control_work() { double s = 0; for (int i = 0; i < 20000; ++i) s += std::sin(i * 1e-3); sink = s; }  // ~"iş"

void report(const char* name, const std::vector<double>& per, double total_ms) {
    std::vector<double> d(per); std::sort(d.begin(), d.end());
    double mean = 0; for (double x : d) mean += x; mean /= d.size();
    std::printf("%-28s ort. periyot %.3f ms | p50 %.3f | p99 %.3f | max %.3f ms | 1000 döngü %.1f ms (ideal 1000)\n",
                name, mean, d[d.size() / 2], d[size_t(d.size() * 0.99)], d.back(), total_ms);
}

int main() {
    const int N = 1000;
    for (int mode = 0; mode < 2; ++mode) {
        std::vector<double> per; per.reserve(N);            // ölçüm vektörü önceden ayrılır: döngüde malloc yok
        auto t0 = Clock::now(), prev = t0, next = t0;
```

```
        for (int k = 0; k < N; ++k) {
            control_work();
            if (mode == 0) std::this_thread::sleep_for(1ms);   // iş süresi + 1 ms -> periyot kayar
            else { next += 1ms; std::this_thread::sleep_until(next); }   // mutlak zamana kadar bekle
            auto now = Clock::now();
            per.push_back(std::chrono::duration<double, std::milli>(now - prev).count()); prev = now;
        }
        double total = std::chrono::duration<double, std::milli>(Clock::now() - t0).count();
        report(mode == 0 ? "sleep_for(1ms)" : "sleep_until(t0 + k*1ms)", per, total);
    }
    auto a = Clock::now(); control_work(); auto b = Clock::now();
    std::printf("tek bir control_work() süresi: %.3f ms\n", std::chrono::duration<double,
std::milli>(b - a).count());
    return 0;
}
```

Çıktı:

```
sleep_for(1ms)              ort. periyot 1.389 ms | p50 1.331 | p99 2.082 | max 4.261 ms | 1000 döngü 1388.6 ms
(ideal 1000)
sleep_until(t0 + k*1ms)     ort. periyot 1.000 ms | p50 0.999 | p99 1.093 | max 1.854 ms | 1000 döngü 1000.1 ms
(ideal 1000)
tek bir control_work() süresi: 0.182 ms
```

`sleep_for` ile 1000 döngü 1389 ms sürdü (%39 yavaş: 0.18 ms iş + uyanma gecikmeleri); `sleep_until` ile 1000.1 ms. Ama `sleep_until`'de bile en kötü periyot 1.85 ms: sıradan bir Linux (hele sanal bir bulut makinesi; bu ölçümler öyle bir makinede alındı) zaman garantisi vermez.

Gerçek robotta: `PREEMPT_RT` çekirdek yaması (Ubuntu'nun gerçek zamanlı çekirdeği), iş parçacığını `SCHED_FIFO` ve yüksek öncelikle çalıştırmak, belleği kilitlemek (`mlockall`), bir CPU çekirdeğini döngüye ayırmak (`isolcpus`, `taskset`), döngüde bellek ayırma/G/Ç yapmamak. Böylece p99 titreşim tipik olarak onlarca mikrosaniyeye iner. SB-1'de hobi servoları zaten 50 Hz komut alıyor; orada en hassas döngü Arduino'daki (Cilt 3), PC tarafında 200–500 Hz yeterli.

Bölüm 7

CMake, Proje Yapısı ve Testler

Tek dosyalık `g++ a.cpp` bir yere kadar; gerçek proje kütüphaneler, uygulamalar ve testlerden oluşur. **CMake** bunları platformdan bağımsız tarif eder; ROS 2'nin `colcon`'u da altta CMake kullanır.

7.1 Yapı

```
sb1_core/
  CMakeLists.txt
  include/sb1/leg.hpp  pid.hpp      # genel arayüz (başka projeler bunu görür)
  src/leg.cpp  pid.cpp              # gerçekleme
  apps/leg_demo.cpp                 # çalıştırılabilir örnek
  tests/test_leg.cpp  test_pid.cpp  # GoogleTest
```

Başlık (`.hpp`) ne yapıldığını, kaynak (`.cpp`) nasıl yapıldığını söyler. `namespace sb1` isim çakışmasını önler. `#pragma once` başlığın iki kez dahil edilmesini engeller.

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(sb1_core VERSION 0.1 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
if(NOT CMAKE_BUILD_TYPE)
  set(CMAKE_BUILD_TYPE Release)
endif()
add_compile_options(-Wall -Wextra -Wpedantic)

# Kütüphane: başlıklar include/, kaynaklar src/
add_library(sb1_core src/leg.cpp src/pid.cpp)
target_include_directories(sb1_core PUBLIC include)

# Komut satırı uygulaması
add_executable(leg_demo apps/leg_demo.cpp)
target_link_libraries(leg_demo PRIVATE sb1_core)

# Testler (GoogleTest). Yol verilmezse GitHub'dan indir.
option(SB1_BUILD_TESTS "Testleri derle" ON)
if(SB1_BUILD_TESTS)
  enable_testing()
  if(GTEST_SOURCE_DIR)
    add_subdirectory(${GTEST_SOURCE_DIR} ${CMAKE_BINARY_DIR}/googletest EXCLUDE_FROM_ALL)
  else()
    include(FetchContent)
    FetchContent_Declare(googletest URL
https://github.com/google/googletest/archive/refs/tags/v1.15.2.zip)
    FetchContent_MakeAvailable(googletest)
  endif()
  add_executable(sb1_tests tests/test_leg.cpp tests/test_pid.cpp)
```

```
  target_link_libraries(sb1_tests PRIVATE sb1_core GTest::gtest_main)
  include(GoogleTest)
  gtest_discover_tests(sb1_tests)
endif()
```

include/sb1/leg.hpp

```cpp
#pragma once
#include <array>
#include <optional>

namespace sb1 {

// Düzlemsel 2-uzuvlu bacak (Cilt 1): kalçaya göre ayak konumu, x ileri, z yukarı
struct LegParams { double l1 = 0.20, l2 = 0.20; };
using Vec2 = std::array<double, 2>;

Vec2 forward(const LegParams& p, double q1, double q2);
// Ters kinematik: ulaşılamazsa boş döner (istisna atmaz; gerçek zamanlı kod için uygun)
std::optional<Vec2> inverse(const LegParams& p, const Vec2& foot);

}  // namespace sb1
```

include/sb1/pid.hpp ve src/pid.cpp

```cpp
#pragma once

namespace sb1 {

class Pid {
public:
    Pid(double kp, double ki, double kd, double u_max) : kp_(kp), ki_(ki), kd_(kd), u_max_(u_max) {}
    double update(double ref, double y, double dt);   // türev ölçümden; kırpmalı anti-windup
    void reset() { i_ = 0; y_prev_ = 0; first_ = true; }
private:
    double kp_, ki_, kd_, u_max_;
    double i_ = 0, y_prev_ = 0;
    bool first_ = true;
};

}  // namespace sb1
```

```cpp
#include "sb1/pid.hpp"
#include <algorithm>

namespace sb1 {

double Pid::update(double ref, double y, double dt) {
    const double e = ref - y;
    const double dy = first_ ? 0.0 : (y - y_prev_) / dt;      // türev ölçümden: hedef basamağında tekme yok
    first_ = false; y_prev_ = y;
    const double u_free = kp_ * e + i_ - kd_ * dy;
    const double u = std::clamp(u_free, -u_max_, u_max_);
    const bool saturating = (u != u_free) && (e * u_free > 0);  // doyma ve hata onu büyütüyor
    if (!saturating) i_ += ki_ * e * dt;
    return u;
}

}  // namespace sb1
```

7.2 Derleme ve test

```
cmake -S . -B build -DCMAKE_BUILD_TYPE =Release
cmake --build build -j4
./build/leg_demo
cd build && ctest --output-on-failure
```

Çıktı:

```
ayak (0.03, -0.30): q1 = -35.37°, q2 =  82.17°
ayak (0.03, -0.35): q1 = -23.67°, q2 =  57.15°
ayak (0.03, -0.39): q1 =  -7.67°, q2 =  24.15°
ayak (0.03, -0.45): ulaşılamaz
```

tests/test_pid.cpp

```cpp
#include <gtest/gtest.h>
#include "sb1/pid.hpp"

TEST(Pid, TwoStepTrace) {                          // Cilt 4A, E2.5 ile aynı sayılar (kd = 0)
    sb1::Pid pid(2.0, 1.0, 0.0, 100.0);
    EXPECT_DOUBLE_EQ(pid.update(1.0, 0.0, 0.1), 2.0);
    EXPECT_DOUBLE_EQ(pid.update(1.0, 0.5, 0.1), 1.1);
}

TEST(Pid, OutputSaturates) {
    sb1::Pid pid(10.0, 0.0, 0.0, 1.0);
    EXPECT_DOUBLE_EQ(pid.update(1.0, 0.0, 0.01), 1.0);
    EXPECT_DOUBLE_EQ(pid.update(-1.0, 0.0, 0.01), -1.0);
}

TEST(Pid, NoWindupWhileSaturated) {                // 1 s doymada kalmak integrali şişirmemeli
    sb1::Pid pid(1.0, 5.0, 0.0, 0.5);
    for (int k = 0; k < 1000; ++k) pid.update(10.0, 0.0, 0.001);
    EXPECT_LT(pid.update(0.0, 0.0, 0.001), 0.5);   // hedef 0 olunca çıkış hemen doymadan çıkmalı
}

TEST(Pid, NoDerivativeKickOnSetpointStep) {
    sb1::Pid pid(0.0, 0.0, 0.4, 1000.0);
    pid.update(0.0, 0.0, 0.001);
    EXPECT_DOUBLE_EQ(pid.update(1.0, 0.0, 0.001), 0.0);   // ölçüm değişmedi -> türev katkısı 0
}
```

Çıktı:

```
Test project .../sb1_core/build
    Start 1: Leg.ForwardStraightDown
1/7 Test #1: Leg.ForwardStraightDown ..............   Passed    0.00 sec
    Start 2: Leg.InverseRoundTrip
2/7 Test #2: Leg.InverseRoundTrip .................   Passed    0.00 sec
    Start 3: Leg.UnreachableReturnsNullopt
3/7 Test #3: Leg.UnreachableReturnsNullopt ........   Passed    0.00 sec
    Start 4: Pid.TwoStepTrace
4/7 Test #4: Pid.TwoStepTrace .....................   Passed    0.00 sec
    Start 5: Pid.OutputSaturates
5/7 Test #5: Pid.OutputSaturates ..................   Passed    0.00 sec
    Start 6: Pid.NoWindupWhileSaturated
6/7 Test #6: Pid.NoWindupWhileSaturated ...........   Passed    0.00 sec
```

```
    Start 7: Pid.NoDerivativeKickOnSetpointStep
7/7 Test #7: Pid.NoDerivativeKickOnSetpointStep ...   Passed    0.00 sec

100% tests passed, 0 tests failed out of 7

Total Test time (real) =   0.02 sec
```

Test gerçekten işe yarıyor mu? Kasıtlı bir hata ekledim: `pid.cpp`'de anti-windup koşulunu silip integrali her zaman güncelledim. Derleyici `-Wall -Wextra` sayesinde "kullanılmayan değişken: saturating" uyarısı verdi ve test düştü: **Çıktı:**

```
Running main() from googletest/googletest/src/gtest_main.cc
tests/test_pid.cpp:19: Failure
Expected: (pid.update(0.0, 0.0, 0.001)) < (0.5), actual: 0.5 vs 0.5
[  FAILED  ] Pid.NoWindupWhileSaturated (0 ms)
[ ==========] 7 tests from 2 test suites ran. (0 ms total)
[  PASSED  ] 6 tests.
```

Buna *mutasyon testi* denir: testlerin bir hatayı yakalayabildiğini ancak hatayı bilerek sokunca anlarsın. İlk yazdığımda bu testi bir hatayla (eski derlemeyi test ederek) "geçti" sandım; temiz bir `build` klasörüyle yeniden yapınca düştü. Ders: şüphelendiğinde `rm -rf build`.

Test edilebilir robot kodu

Donanıma dokunan kodu (seri port, servo) ince bir katmanda tut; hesap yapan kodu (IK, PID, kestirici) saf fonksiyonlar/sınıflar olarak ayır. Böylece hesap kodunu robotsuz, her commit'te, saniyeler içinde test edersin. Cilt 3'teki `arduino_mock.hpp` bunun küçük bir örneğiydi.

Bölüm 8

Robotik için Python

Python; analiz, prototip, simülasyon betikleri, veri işleme ve öğrenme (Cilt 6 Bölüm 5) için. C++ ise robotun üstünde, hızlı döngülerde. İkisini `pybind11` ile bağlamak yaygın: C++ kütüphanesi (`sb1_core`) Python'dan çağrılır.

8.1 Ortam

```
python3 -m venv .venv && source .venv/bin/activate   # projeye özel ortam
pip install numpy scipy matplotlib pytest mujoco
pip freeze > requirements.txt                          # tekrarlanabilirlik
```

8.2 Alışkanlıklar ve tuzaklar

- **Vektörleştir:** NumPy işlemleri dizinin tamamında C hızında çalışır; Python `for` döngüsü her eleman için yorumlayıcı maliyeti öder.
- **`@dataclass`** parametre kümeleri için; `frozen =True` kazara değişikliği engeller. Tip ipuçları (`def f(q: np.ndarray) -> float`) okunabilirlik ve `mypy` denetimi için.
- `if __name__ == "__main__":` ile betiği hem modül hem program yap.

Python

```python
"""y7_python.py — robotik için iyi Python alışkanlıkları: dataclass, tip ipuçları, vektörleştirme, tuzaklar."""
from __future__ import annotations
from dataclasses import dataclass, field
import time
import numpy as np

@dataclass(frozen=True)                       # frozen: yanlışlıkla değiştirilemez parametre kümesi
class LegParams:
    l1: float = 0.20
    l2: float = 0.20

@dataclass
class Robot:
    name: str
    joint_names: list[str] = field(default_factory=list)   # DOĞRU: her nesneye yeni liste

def forward_loop(p: LegParams, q: np.ndarray) -> np.ndarray:
    out = np.empty((len(q), 2))
    for i, (q1, q2) in enumerate(q):          # Python döngüsü: her eleman için yorumlayıcı maliyeti
        out[i] = (p.l1 * np.sin(q1) + p.l2 * np.sin(q1 + q2), -p.l1 * np.cos(q1) - p.l2 * np.cos(q1 + q2))
    return out
```

```python

def forward_vec(p: LegParams, q: np.ndarray) -> np.ndarray:
    q1, q2 = q[:, 0], q[:, 1]                # vektörleştirilmiş: tüm dizi C'de tek seferde
    return np.c_[p.l1 * np.sin(q1) + p.l2 * np.sin(q1 + q2), -p.l1 * np.cos(q1) - p.l2 * np.cos(q1 + q2)]

p = LegParams()
q = np.random.default_rng(0).uniform([-0.8, 0.1], [0.8, 2.0], (100_000, 2))
t0 = time.perf_counter(); a = forward_loop(p, q); t1 = time.perf_counter(); b = forward_vec(p, q); t2 = time.perf_counter()
print(f"100 000 FK: döngü {1000 * (t1 - t0):.0f} ms, vektörleştirilmiş {1000 * (t2 - t1):.1f} ms "
      f"(~{(t1 - t0) / (t2 - t1):.0f} kat), fark {np.abs(a - b).max():.1e}")

# Tuzak 1: değiştirilebilir varsayılan argüman
def add_joint_bad(name, joints=[]):          # liste fonksiyon TANIMLANIRKEN bir kez oluşturulur!
    joints.append(name); return joints
print("tuzak 1:", add_joint_bad("diz"), add_joint_bad("kalca"), "<- ikinci çağrı ilkinin listesini görüyor")
r1, r2 = Robot("SB-1"), Robot("SB-2"); r1.joint_names.append("diz")
print("dataclass default_factory:", r1.joint_names, r2.joint_names)

# Tuzak 2: kayan nokta eşitliği
x = 0.1 + 0.2
print("tuzak 2: 0.1 + 0.2 == 0.3 ?", x == 0.3, "| np.isclose:", np.isclose(x, 0.3))

# Tuzak 3: NumPy görünümü (view) ile kopya
a = np.zeros(4); v = a[1:3]; v[:] = 7
print("tuzak 3: dilim bir görünüm, orijinal değişti:", a, "| güvenli: a[1:3].copy()")

# Tuzak 4: dondurulmuş parametre değiştirilemez
try:
    p.l1 = 0.3
except Exception as e:
    print("tuzak 4 (iyi anlamda): frozen dataclass ->", type(e).__name__)
```

Çıktı:

```
100 000 FK: döngü 372 ms, vektörleştirilmiş 12.1 ms (~31 kat), fark 0.0e+00
tuzak 1: ['diz', 'kalca'] ['diz', 'kalca'] <- ikinci çağrı ilkinin listesini görüyor
dataclass default_factory: ['diz'] []
tuzak 2: 0.1 + 0.2 == 0.3 ? False | np.isclose: True
tuzak 3: dilim bir görünüm, orijinal değişti: [0. 7. 7. 0.] | güvenli: a[1:3].copy()
tuzak 4 (iyi anlamda): frozen dataclass -> FrozenInstanceError
```

100 000 ileri kinematik hesabında vektörleştirme yaklaşık 30 kat hızlı (bu makinede 372 ms'ye karşı 12 ms).

NumPy neden hızlı? Python'da her `a + b` işlemi: nesnelerin türünü kontrol et, uygun metodu bul, yeni bir `float` nesnesi oluştur (öbekte), referans sayacını güncelle: yüzlerce makine komutu. NumPy dizisi ise aynı türden sayıların bitişik bir bellek bloğudur; `np.sin(a)` bu kontrolleri *bir kez* yapar, sonra derlenmiş bir C döngüsü bloğu baştan sona işler (çoğu zaman SIMD komutlarıyla birkaç sayıyı aynı anda). Ölçtüğümüz 30 kat bu farkın sonucu. Vektörleştiremediğin algoritmalarda (ardışık bağımlı döngüler: simülasyon) çözüm C++ (Cilt 1: 1000 kat) veya Numba/pybind11.

8.3 pytest

proje/py_sb1/tests/test_leg.py

```python
import numpy as np
import pytest
from leg import forward, inverse

def test_straight_down():
    assert forward(0.0, 0.0) == pytest.approx((0.0, -0.4))

@pytest.mark.parametrize("q1", np.linspace(-0.8, 0.8, 5))
@pytest.mark.parametrize("q2", np.linspace(0.1, 2.0, 5))
def test_round_trip(q1, q2):
    assert inverse(*forward(q1, q2)) == pytest.approx((q1, q2), abs=1e-9)

def test_unreachable():
    assert inverse(0.0, -0.45) is None
```

Çıktı:

```
.........................                                [100%]
27 passed in 0.18s
```

`parametrize` ile $5 \times 5 = 25$ eklem açısı çiftinde "IK(FK(q)) eşittir q" kontrol edildi; artı 2 test, toplam 27.

Bölüm 9

SB-1'in MuJoCo Modeli (MJCF)

MuJoCo modeli bir XML dosyasıdır: gövdeler (`body`) iç içe ağaç oluşturur; her gövdenin ataleti (`inertial`), ebeveynine göre eklemi (`joint`), çarpışma geometrisi (`geom`) vardır; `actuator` eklemleri sürer, `sensor` ölçer. Menagerie'deki `robotis_op3` ve `berkeley_humanoid` modellerini açıp okumak en iyi öğretmen.

Bu bölümde XML'i elle değil, **Python'dan üretiyoruz**: kütle ve boyutlar tek yerde, ataletler Cilt 1'in kutu formülüyle hesaplanıyor, iki bacak aynı fonksiyonla simetrik kuruluyor. Parametreler Cilt 6 Bölüm 4'ün sonucuna göre **hafif** seçildi (toplam 1.28 kg): böylece MG996R diz, tek ayakta diz–ayak yatay mesafesi 8.6 cm'ye kadar (kabaca 50° diz bükümüne kadar) ağırlığı taşıyabiliyor; ama bu durma torku, sürekli çalışma için daha temkinli olmak gerekir.

Denenmedi

MuJoCo bu ortamda kurulu olmadığından model burada yüklenip simüle edilmedi; yalnızca XML'in geçerli olduğu, 10 eklem/aktüatör ve kütle toplamı kontrol edildi. Kendi bilgisayarında `python -m mujoco.viewer --mjcf =sb1.xml` ile aç; ilk kontrol listesi: robot yere doğru yükseklikte mi duruyor, eklem eksenleri ve sınırları doğru yönde mi, ayaklar yere giriyor mu, `kp` ile servo davranışı Cilt 3'teki ölçümlere benziyor mu.

Python: MJCF üretici

```python
"""y8_mjcf.py — SB-1'in MuJoCo (MJCF) modelini Python'dan ÜRETİR: kütle/atalet Cilt 1 formülleriyle hesaplanır,
XML tek kaynaktan (parametrelerden) çıkar. Elle 300 satır XML yazmak yerine kodla üretmek hataları azaltır.
NOT: Bu ortamda MuJoCo kurulu olmadığı için model burada simüle EDİLMEDİ; sadece XML geçerliliği, kütle toplamı ve atalet değerleri kontrol edildi. Kendi bilgisayarında:  python -m mujoco.viewer --mjcf =sb1.xml"""
import xml.etree.ElementTree as ET
from xml.dom import minidom

def box_inertia(m, a, b, c):
    """Kütle merkezinden geçen eksenlere göre katı kutu (a, b, c kenarları x, y, z): Cilt 1 Bölüm 6."""
    return (m * (b * b + c * c) / 12, m * (a * a + c * c) / 12, m * (a * a + b * b) / 12)

# Parametreler: Cilt 6 Bölüm 4'teki tork sonucuna göre HAFİF tasarım (toplam ~1.2 kg)
P = dict(torso=(0.40, (0.10, 0.14, 0.12)),   # kütle [kg], kutu boyutları [m] (gövde + Arduino + pil)
         hip=(0.06, (0.04, 0.04, 0.04)),     # kalça yaw/roll arası bağlantı parçası (servo dahil)
         thigh=(0.15, (0.04, 0.04, 0.20)),
         shank=(0.12, (0.04, 0.04, 0.20)),
         foot=(0.05, (0.08, 0.05, 0.01)))
SERVO_TAU = 1.08                              # MG996R durma torku, 6 V (datasheet: 11 kgf·cm)
KP = 8.0                                      # konum servo kazancı [N·m/rad] (Cilt 3'ün servo modeline göre ayarlanmalı)

def f3(v): return " ".join(f"{x:.6g}" for x in v)

def body(parent, name, pos, part, com, joint=None):
    b = ET.SubElement(parent, "body", name=name, pos=f3(pos))
```

```
    m, (a, bb, c) = P[part]
    ET.SubElement(b, "inertial", pos=f3(com), mass=f"{m:.4g}", diaginertia=f3(box_inertia(m, a, bb, c)))
    if joint:
        jn, axis, rng = joint
        ET.SubElement(b, "joint", name=jn, axis=axis, range=f3(rng))
    ET.SubElement(b, "geom", type="box", size=f3((a / 2, bb / 2, c / 2)), pos=f3(com))
    return b

root = ET.Element("mujoco", model="sb1")
ET.SubElement(root, "compiler", angle="radian")
ET.SubElement(root, "option", timestep="0.001")
d = ET.SubElement(root, "default")
ET.SubElement(d, "joint", damping="0.05", armature="0.002", frictionloss="0.02")
ET.SubElement(d, "geom", contype="1", conaffinity="1", friction="0.8 0.02 0.001", rgba="0.4 0.6 0.8 1")
ET.SubElement(d, "position", kp=f"{KP}", forcelimited="true", forcerange=f"{-SERVO_TAU} {SERVO_TAU}")
wb = ET.SubElement(root, "worldbody")
ET.SubElement(wb, "light", pos="0 0 2", dir="0 0 -1")
ET.SubElement(wb, "geom", name="floor", type="plane", size="3 3 0.1", rgba="0.85 0.85 0.85 1")
torso = body(wb, "torso", (0, 0, 0.51), "torso", (0, 0, 0.03))
ET.SubElement(torso, "freejoint", name="root")
ET.SubElement(torso, "site", name="imu", pos="0 0 0.03")
joints = []
for side, y in (("sol", 0.05), ("sag", -0.05)):
    h = body(torso, f"{side}_kalca_yaw", (0, y, -0.04), "hip", (0, 0, -0.02), (f"{side}_kalca_yaw", "0 0 1", (-0.5, 0.5)))
    r = body(h, f"{side}_kalca_roll", (0, 0, -0.04), "hip", (0, 0, 0), (f"{side}_kalca_roll", "1 0 0", (-0.4, 0.4)))
    t = body(r, f"{side}_uyluk", (0, 0, 0), "thigh", (0, 0, -0.10), (f"{side}_kalca_pitch", "0 1 0", (-1.5, 1.0)))
    s = body(t, f"{side}_baldir", (0, 0, -0.20), "shank", (0, 0, -0.10), (f"{side}_diz", "0 1 0", (0.0, 2.3)))
    f = body(s, f"{side}_ayak", (0, 0, -0.20), "foot", (0.01, 0, -0.015), (f"{side}_bilek_pitch", "0 1 0", (-1.0, 1.0)))
    joints += [f"{side}_kalca_yaw", f"{side}_kalca_roll", f"{side}_kalca_pitch", f"{side}_diz", f"{side}_bilek_pitch"]
act = ET.SubElement(root, "actuator")
for j in joints:
    ET.SubElement(act, "position", name=j, joint=j)
sens = ET.SubElement(root, "sensor")
ET.SubElement(sens, "gyro", site="imu"); ET.SubElement(sens, "accelerometer", site="imu")

xml = minidom.parseString(ET.tostring(root)).toprettyxml(indent="  ")
open("sb1.xml", "w").write(xml)
ET.fromstring(xml)                              # yeniden ayrıştır: biçim hatası varsa burada patlar
total = P["torso"][0] + 2 * (2 * P["hip"][0] + P["thigh"][0] + P["shank"][0] + P["foot"][0])
print(f"sb1.xml yazıldı: {len(xml.splitlines())} satır, {len(joints)} eklem/aktüatör, toplam kütle {total:.2f} kg")
print("uyluk ataleti (Ixx, Iyy, Izz) [kg·m²]:", f3(box_inertia(*((P['thigh'][0],) + P['thigh'][1]))))
print(f"tek ayak üstünde tüm ağırlık {total * 9.81:.1f} N; diz torku sınırı {SERVO_TAU} N·m için "
      f"diz-ayak yatay mesafesi ≤ {100 * SERVO_TAU / (total * 9.81):.1f} cm (Cilt 6 Bölüm 4)")
print("\n".join(xml.splitlines()[:14]) + "\n  ...")
```

Çıktı:

```
sb1.xml yazıldı: 86 satır, 10 eklem/aktüatör, toplam kütle 1.28 kg
uyluk ataleti (Ixx, Iyy, Izz) [kg·m²]: 0.00052 0.00052 4e-05
tek ayak üstünde tüm ağırlık 12.6 N; diz torku sınırı 1.08 N·m için diz-ayak yatay mesafesi ≤ 8.6 cm (Cilt 6 Bölüm 4)
<?xml version ="1.0" ?>
<mujoco model ="sb1">
```

```
<compiler angle ="radian"/>
<option timestep ="0.001"/>
<default>
  <joint damping ="0.05" armature ="0.002" frictionloss ="0.02"/>
  <geom contype ="1" conaffinity ="1" friction ="0.8 0.02 0.001" rgba ="0.4 0.6 0.8 1"/>
  <position kp ="8.0" forcelimited ="true" forcerange ="-1.08 1.08"/>
</default>
<worldbody>
  <light pos ="0 0 2" dir ="0 0 -1"/>
  <geom name ="floor" type ="plane" size ="3 3 0.1" rgba ="0.85 0.85 0.85 1"/>
  <body name ="torso" pos ="0 0 0.51">
    <inertial pos ="0 0 0.03" mass ="0.4" diaginertia ="0.00113333 0.000813333 0.000986667"/>
...
```

Notlar: `position` aktüatörü MuJoCo'da bir PD konum servosu modelidir ($\tau = k_p(q_{\text{hedef}} - q)$, `forcerange` ile kırpılı); hobi servosunun iç denetleyicisine kabaca benzer. Cilt 3'teki gecikme ve hız sınırını eklemek için kontrol döngüsünde komutu 20 ms geciktir ve eğim sınırla. `armature` motorun yansıyan rotor ataletidir ($J_m N^2$; Cilt 3'teki $J_m = 2 \cdot 10^{-7}$, $N = 195$ ile ≈ 0.008; buradaki 0.002 kaba bir başlangıç değeri, sistem tanılama ile ayarlanmalı).

Bölüm 10

ROS 2

ROS 2, robot yazılımını birbirinden bağımsız **düğümlere** (node) böler; düğümler **konular** (topic, yayın/abone), **servisler** (istek/cevap) ve **eylemler** (uzun süren, geri bildirimli görevler) ile konuşur. Altta DDS ara katmanı çalışır. Kurulum: Ubuntu 24.04 + ROS 2 Jazzy (veya 22.04 + Humble).

Denenmedi

Bu bölümdeki kod ROS 2 kurulu olmadığı için burada derlenmedi; rclcpp'nin Humble/Jazzy API'sine göre yazıldı. Küçük derleme hataları çıkarsa hata mesajını oku: genelde eksik bir `find_package` veya başlık yolu.

10.1 Temel komutlar

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker          # bir terminalde
ros2 topic list; ros2 topic echo /chatter; ros2 topic hz /chatter
ros2 node info /talker; ros2 param list
mkdir -p ~/ws/src && cd ~/ws && colcon build --symlink-install && source install/setup.bash
ros2 launch sb1_ros leg.launch.py
ros2 topic pub /foot_target geometry_msgs/msg/Point "{x: 0.03, y: 0.0, z: -0.35}"
```

10.2 SB-1 IK düğümü

C++ (rclcpp)

```cpp
// leg_ik_node.cpp — ROS 2 (rclcpp) düğümü: hedef ayak konumunu dinler, IK ile eklem komutu yayınlar.
// DİKKAT: Bu dosya bu ortamda DERLENMEDİ (ROS 2 kurulu değil). Humble/Jazzy rclcpp API'sine göre yazıldı.
#include <chrono>
#include <memory>
#include <geometry_msgs/msg/point.hpp>
#include <rclcpp/rclcpp.hpp>
#include <sensor_msgs/msg/joint_state.hpp>
#include "sb1/leg.hpp"                                  // Bölüm 6'daki sb1_core kütüphanesi

using namespace std::chrono_literals;

class LegIkNode : public rclcpp::Node {
public:
    LegIkNode() : Node("leg_ik") {
        params_.l1 = declare_parameter("l1", 0.20);     // ros2 run ... --ros-args -p l1:=0.21
        params_.l2 = declare_parameter("l2", 0.20);
        pub_ = create_publisher<sensor_msgs::msg::JointState>("joint_command", 10);
        sub_ = create_subscription<geometry_msgs::msg::Point>(
            "foot_target", 10, [this](geometry_msgs::msg::Point::SharedPtr msg) { target_ = *msg; });
```

```
        timer_ = create_wall_timer(10ms, [this] { tick(); });   // 100 Hz
    }

private:
    void tick() {
        auto q = sb1::inverse(params_, {target_.x, target_.z});
        if (!q) { RCLCPP_WARN_THROTTLE(get_logger(), *get_clock(), 1000, "hedef erişim dışında");
return; }
        sensor_msgs::msg::JointState js;
        js.header.stamp = now();
        js.name = {"sol_kalca_pitch", "sol_diz"};
        js.position = {(*q)[0], (*q)[1]};
        pub_->publish(js);
    }
    sb1::LegParams params_;
    geometry_msgs::msg::Point target_{};
    rclcpp::Publisher<sensor_msgs::msg::JointState>::SharedPtr pub_;
    rclcpp::Subscription<geometry_msgs::msg::Point>::SharedPtr sub_;
    rclcpp::TimerBase::SharedPtr timer_;
};

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<LegIkNode>());
    rclcpp::shutdown();
    return 0;
}
```

launch dosyası

```
# leg.launch.py — DİKKAT: bu ortamda çalıştırılmadı.
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package="sb1_ros", executable="leg_ik_node", name="leg_ik",
             parameters=[{"l1": 0.20, "l2": 0.20}], output="screen"),
    ])
```

Dikkat edilecekler: düğüm sınıfı `rclcpp::Node`'dan türer; yayıncı, abone ve zamanlayıcı `shared_ptr` olarak üyelerde tutulur (yoksa kapsam bitince yok olur: Bölüm 3); geri çağırmalar lambda; parametreler `declare_parameter` ile; hesap kodu (`sb1::inverse`) ROS'tan bağımsız kütüphanede, böylece Bölüm 7'deki testler ROS'suz çalışır.

10.3 ros2_control ve hızlı döngü

Konu tabanlı haberleşme 10–100 Hz için rahat; 1 kHz eklem kontrolü için değil. `ros2_control` donanımı bir *donanım arayüzü* (`read()`/`write()`) ile soyutlar, denetleyiciler (ör. `joint_trajectory_controller`, kendi yürüme denetleyicin) aynı gerçek zamanlı döngüde, mesajlaşma olmadan çalışır. SB-1 için yol: Arduino ile seri haberleşen bir `SystemInterface` eklentisi, üstünde Cilt 6'daki DCM/WBC'yi içeren bir denetleyici. Simülasyonda aynı denetleyiciyi MuJoCo'ya bağlayan köprüler mevcut; böylece simülasyon ve gerçek robot aynı kodu çalıştırır.

Bölüm 11

Sorular

(K): kod okuma; kâğıtta çıktıyı tahmin et.

Y1–2 — Linux ve Git

Y1.1 `./robot > log.txt 2>&1` ne yapar? `2>&1` olmasaydı hata mesajları nereye giderdi?

Y1.2 Arduino'yu taktın, `/dev/ttyACM0` açılırken "Permission denied". İki doğru çözüm ve bir yanlış çözüm söyle.

Y1.3 `grep -rn "Kp" src/ | wc -l` ne üretir?

Y2.1 `git add` ile `git commit` arasındaki fark nedir? Neden iki adım?

Y2.2 Bir PID değişikliği için kötü ve iyi bir commit mesajı yaz.

Y2.3 `build/` klasörü neden depoya girmemeli?

Y3–4 — C++

Y3.1 RAII'yi bir cümleyle tanımla. C'deki `goto cleanup` kalıbıyla ilişkisi?

Y3.2 (K) `auto s = std::make_unique<Servo>(1); auto t = std::move(s);` sonrası `s` ne durumda? `s->id` yazarsan ne olur?

Y3.3 (K) `shared_ptr a`; içte `b = a`; iç kapsam bitince ve `a.reset()` sonrası sayaçlar ve silinme anı?

Y3.4 `SerialPort(const SerialPort&) = delete;` neden? Yazılmasaydı `std::vector<SerialPort>` kopyalandığında ne olurdu?

Y4.1 (K) Halka tampon $N = 5$, sırayla 1, 1, 1, 9, 1 eklendi. Ortalama ve medyan?

Y4.2 (K) `std::count_if(v.begin(), v.end(), [](int x){return x > 2;})`, `v ={1,5,2,7,3}`?

Y4.3 Gerçek zamanlı döngü içinde neden `push_back` ve `printf` sakıncalı?

Y5–6 — Eşzamanlılık ve gerçek zaman

Y5.1 Veri yarışı nedir? Korumasız sayaç neden 4004 yerine 2002 verebilir (bir senaryo yaz)?

Y5.2 Bir `bool estop` bayrağı için mutex mi atomic mi?

Y6.1 İş 0.18 ms, `sleep_for(1ms)` ve uyanma gecikmesi ortalama 0.2 ms. 1000 döngü kaç ms sürer? `sleep_until` ile?

Y6.2 "Gerçek zamanlı = hızlı" ifadesi neden yanlış?

Y7–10 — Proje, Python, MJCF, ROS 2

Y7.1 Başlık ve kaynak dosyası ayrımı neden? `target_include_directories(... PUBLIC include)`'daki `PUBLIC` ne demek?

Y7.2 Mutasyon testi nedir; bu ciltte neyi gösterdi?

Y8.1 (K) `def f(x, l =[]): l.append(x); return l. f(1); print(f(2))` ne yazdırır? Düzelt.

Y9.1 Uyluk 0.15 kg, $0.04 \times 0.04 \times 0.20$ m kutu: kütle merkezinden geçen yatay eksene göre atalet? Kalça eksenine (uç) göre?

Y10.1 Konu, servis ve eylem arasındaki fark; SB-1'den birer örnek.

Bölüm 12

Cevaplar

Y1–2

Y1.1 Standart çıktıyı `log.txt`'ye yazar, standart hatayı (2) da standart çıktının (1) gittiği yere yönlendirir. Olmasaydı hatalar terminale düşerdi, dosyada olmazdı.

Y1.2 Doğru: kullanıcıyı `dialout` grubuna ekleyip yeniden giriş yapmak; bir udev kuralıyla cihaza izin vermek. Yanlış: programı `sudo` ile çalıştırmak (veya `chmod 777 /dev/ttyACM0`, her takışta sıfırlanır).

Y1.3 `src/` altında "Kp" geçen satır sayısı.

Y2.1 `add` değişikliği bir sonraki anlık görüntüye (sahne) koyar; `commit` sahnedekini kaydeder. İki adım, ilgisiz değişiklikleri ayrı commit'lere bölmeni sağlar.

Y2.2 Kötü: "düzeltme". İyi: "pid: türevi ölçümden al; hedef basamağında 400 N · m tekme vardı (Cilt 4A E2.4)".

Y2.3 Makineye özgü, yeniden üretilebilir, büyük ve sürekli değişen dosyalar; depoyu şişirir, çakışma üretir.

Y3–4

Y3.1 Kaynağın ömrü nesnenin ömrüne bağlıdır: kurucu alır, yıkıcı bırakır. `goto cleanup`'ın elle yaptığını (her çıkışta temizlik) derleyici garanti eder.

Y3.2 `s` boş (`nullptr`); `s->id` tanımsız davranış, büyük olasılıkla çökme.

Y3.3 İçeride 2, iç kapsam bitince 1, `reset` ile 0 ve o anda silinir.

Y3.4 İki nesne aynı portu temsil eder ve ikisinin yıkıcısı da kapatmaya çalışır (çift kapatma). `= delete` ile kopyalayan kod derlenmez.

Y4.1 Ortalama $13/5 = 2.6$; sıralı $1, 1, 1, 1, 9 \Rightarrow$ medyan 1.

Y4.2 5, 7, 3: **3**.

Y4.3 `push_back` kapasite dolunca bellek ayırır (süresi belirsiz, kilit alabilir); `printf` G/Ç yapar ve bloklayabilir. İkisi de döngünün en kötü süresini öngörülemez yapar.

Y5–6

Y5.1 İki iş parçacığı aynı veriye eşzamanlı, en az biri yazarak, senkronizasyonsuz erişir. Senaryo: A `t =racy` (0) okur, B `t =racy` (0) okur, A 1 yazar, B 1 yazar: iki artırma, sonuç 1. Bu her adımda olursa yarısı kaybolur.

Y5.2 `std::atomic<bool>`: tek değişken, kilitsiz, gerçek zamanlı döngüyü bekletmez.

Y6.1 $1000 \cdot (0.18 + 1 + 0.2) = 1380$ ms (ölçülen 1389). `sleep_until`: ≈ 1000 ms (iş + gecikme 1 ms'nin altında kaldıkça).

Y6.2 Gerçek zamanlılık, *son tarihin her seferinde* karşılanması (öngörülebilirlik, en kötü durum) demektir; ortalaması hızlı ama arada bir 5 ms takılan bir döngü gerçek zamanlı değildir.

Y7–10

Y7.1 Başlık arayüzü (ne), kaynak gerçeklemeyi (nasıl) taşır: kullanıcılar sadece başlığı görür, değişiklikte sadece ilgili kaynak yeniden derlenir. `PUBLIC`: bu dizin hem kütüphanenin kendisine hem de ona bağlanan hedeflere (testler, uygulama) aktarılır.

Y7.2 Koda bilerek hata sokup testlerin onu yakalayıp yakalamadığını görmek. Anti-windup silinince `NoWindupWhileSaturated` düştü; derleyici uyarısı da ipucu verdi.

Y8.1 `[1, 2]`: varsayılan liste bir kez oluşturulur, çağrılar arasında paylaşılır. Düzeltme: `def f(x, l =None): l = [] if l is None else l; ...`

Y9.1 $I_{\text{cm}} = \frac{m}{12}(b^2 + c^2) = \frac{0.15}{12}(0.0016 + 0.04) = 5.2 \cdot 10^{-4}$ kg·m^2. Uca göre (paralel eksen): $5.2 \cdot 10^{-4} + 0.15 \cdot 0.1^2 = 2.02 \cdot 10^{-3}$.

Y10.1 Konu: sürekli veri akışı, yayın/abone (`/imu`, `/joint_states`). Servis: kısa istek–cevap ("kalibrasyon sıfırını kaydet"). Eylem: uzun süren, geri bildirimli, iptal edilebilir görev ("şu noktaya yürü").